

UNIVERSIDAD EUROPEA DE VALENCIA
Grado en Física

Subject: Introduction to artificial intelligence

Activity 2 - Computing Higgs masses using Gradient Descent algorithms

Author:
Pau Gallego Cisteró

Date:
February 27, 2026

CONTENTS

1	Summary and objectives	2
2	Principles	2
2.1	Physical Basis: The Higgs Mechanism	2
2.2	Beyond the Standard Model: Two Higgs Doublet Model (THDM)	2
2.3	Numerical Optimization: Gradient Descent Variants	3
2.4	Evaluation: Convergence and Stability	3
3	Calculations	3
3.1	Part 1: Standard Model Higgs (1D)	3
3.2	Part 2: Two Higgs Doublet Model (2D)	4
4	Analysis	7
4.1	Part 1: Standard Model Higgs (1D)	7
4.2	Part 2: Two Higgs Doublet Model (2D)	8
4.3	Combined Graphs	10
4.4	Comparison of Numerical and Analytical Results	11
4.5	Errors	11
5	Conclusions	12
6	Code	12
6.1	Part 1:	12
6.2	Part 2:	16
7	AI usage	22

1. SUMMARY AND OBJECTIVES

This activity explores numerical optimization in the context of high-energy physics by implementing and comparing three Gradient Descent variants: basic Gradient Descent (GD), Gradient Descent with Momentum (GDM), and Nesterov Accelerated Gradient (NAG). By minimizing the Higgs potential within the Standard Model and the Two Higgs Doublet Model (THDM), the exercise demonstrates how iterative algorithms can determine vacuum expectation values (VEVs) to calculate boson mass eigenstates. The project bridges the gap between theoretical calculus and artificial intelligence, requiring both the implementation of algorithms from scratch and the analytical validation of results through the second partial derivative test.

2. PRINCIPLES

The theoretical foundation of this activity is built upon two pillars: the physical modeling of particle mass generation and the numerical framework of first-order optimization algorithms.

2.1. Physical Basis: The Higgs Mechanism

In the Standard Model, the Higgs boson is a fundamental scalar particle that provides mass to other particles through spontaneous symmetry breaking. The minimization of the Higgs potential determines the vacuum expectation value (VEV) v . The Standard Model Higgs potential is expressed as:

$$V(\phi) = -\mu^2|\phi|^2 + \lambda|\phi|^4; \quad (1)$$

where μ and λ are parameters that determine the shape of the potential. This potential has a minimum at $\langle\phi\rangle = v/\sqrt{2}$, which directly determines the Higgs mass:

$$m_h^2 = 2\lambda v^2. \quad (2)$$

Determining the exact value of ϕ that minimizes $V(\phi)$ is essential for calculating the physical mass of the boson.

2.2. Beyond the Standard Model: Two Higgs Doublet Model (THDM)

To explore richer phenomenology, the Higgs sector can be extended to include two scalar fields, Φ_1 and Φ_2 . This modification leads to multiple Higgs bosons with distinct mass eigenstates. The potential in the THDM is given by:

$$V(\Phi_1, \Phi_2) = -\alpha_1|\Phi_1|^2 - \alpha_2|\Phi_2|^2 + \frac{\beta_1}{2}|\Phi_1|^4 + \frac{\beta_2}{2}|\Phi_2|^4 + \beta_3|\Phi_1|^2|\Phi_2|^2. \quad (3)$$

Minimizing this multi-variable potential allows for the calculation of the masses for CP-even bosons (h^0, H^0) and charged bosons ($H^\pm$) based on the identified vacuum expectation values v_1 and v_2 .

2.3. Numerical Optimization: Gradient Descent Variants

To find the minima of these potentials numerically, we use Gradient Descent, an iterative algorithm that updates field values in the opposite direction of the gradient.

- **Basic Gradient Descent (GD):** Updates the field using the rule $\phi_{t+1} = \phi_t - \eta \nabla V(\phi_t)$, where η is the learning rate.
- **GD with Momentum (GDM):** Introduces a momentum term γ to accelerate convergence and avoid local minima by considering previous update directions.
- **Nesterov Accelerated Gradient (NAG):** An advanced variant that calculates the gradient at a "look-ahead" position, improving stability and speed.

2.4. Evaluation: Convergence and Stability

The performance of these algorithms is evaluated based on their ability to converge to the theoretical expectation $\langle \phi_{th} \rangle$.

1. **Convergence Criteria:** The algorithm continues until the difference between the computed ϕ and the theoretical value is less than a predefined threshold (e.g., 1 GeV).
2. **Stability Analysis:** The second partial derivative test is applied to the identified minima. A positive determinant of the Hessian matrix ($\det(H) > 0$) confirms that the algorithm has settled in a stable local minimum rather than a saddle point.

By comparing the number of iterations required for each method, we can determine the most efficient approach for high-energy physics optimization problems.

3. CALCULATIONS

In this part we will show and make the calculations requested by hand on *Part 3* of the Activity guideline.

3.1. Part 1: Standard Model Higgs (1D)

The objective function for this analysis is defined by the Higgs potential, which acts as our cost function,

$$V(\phi) = -\mu^2 |\phi|^2 + \lambda |\phi|^4 \quad (\text{cost function}).$$

We first derive the gradient of the potential. This expression is essential for the subsequent algorithmic implementation and serves as the basis for the analytical verification of the minimum,

$$\frac{dV}{d\phi} = -2\mu^2 \phi + 4\lambda \phi^3 \quad \text{with} \quad \begin{cases} \mu = 88.39 \text{ GeV} \\ \lambda = 0.129 \end{cases}$$

To identify the stationary points, we perform a minimization by setting the first derivative to zero, $\frac{dV}{d\phi} = 0$,

$$\Rightarrow 2\phi(-\mu^2 + 2\lambda\phi^2) = 0.$$

Evaluating this expression yields two distinct solutions for the field:

$$\longrightarrow \phi = 0$$

$$\longrightarrow \phi^2 = \frac{\mu^2}{2\lambda} \Rightarrow \phi_{min} = \sqrt{\frac{\mu^2}{2\lambda}} = \frac{\mu}{\sqrt{2\lambda}}$$

To determine the nature of these critical points and confirm the stability of the vacuum state, we apply the second derivative test,

$$\frac{d^2V}{d\phi^2} = -2\mu^2 + 12\lambda\phi^2,$$

Evaluating the second-order derivative at the non-trivial stationary point ϕ_{min} gives:

$$\frac{d^2V}{d\phi^2} = -2\mu^2 + 12\lambda \left(\frac{\mu^2}{2\lambda} \right) = -2\mu^2 + 6\mu^2 = 4\mu^2.$$

Given that $\mu = 88.39$ GeV, it follows that $\mu^2 > 0$. Consequently, the second derivative evaluated at the critical point, $V''(\phi_{min}) = 4\mu^2$, is strictly positive, satisfying the second-order sufficiency condition for a local minimum. Given the symmetry and the asymptotic behavior of the Higgs potential, this point constitutes the stable global minimum of the system. Having validated the stability of this vacuum state, we proceed to determine the numerical magnitudes of the field expectation value and the associated Higgs boson mass, m_h .

$$\Rightarrow \phi_{min} = \frac{88.39}{\sqrt{2 \cdot 0.129}} = 174.0176 \text{ GeV.}$$

$$\text{We know } \langle \phi \rangle = \frac{v}{\sqrt{2}} \Rightarrow v = 174.0176 \cdot \sqrt{2} \approx 246.098 \text{ GeV.}$$

$$\rightarrow m_h^2 = 2\lambda v^2 = 2 \cdot 0.129 \cdot (246.098)^2 = 15625.57 \Rightarrow$$

$$m_h = \sqrt{15625.57} \approx 125 \text{ GeV.}$$

3.2. Part 2: Two Higgs Doublet Model (2D)

In this scenario, the system is characterized by a potential involving two different variables, defined as

$$V(\Phi_1, \Phi_2) = -\alpha_1|\Phi_1|^2 - \alpha_2|\Phi_2|^2 + \frac{\beta_1}{2}|\Phi_1|^4 + \frac{\beta_2}{2}|\Phi_2|^4 + \beta_3|\Phi_1|^2|\Phi_2|^2.$$

To analyze the stability of the vacuum states and classify the stationary points, it is necessary to evaluate the second-order partial derivatives:

$$\rightarrow V_{11} = \frac{\partial^2 V}{\partial \Phi_1^2}$$

$$\rightarrow V_{22} = \frac{\partial^2 V}{\partial \Phi_2^2}$$

$$\rightarrow V_{12} = V_{21} = \frac{\partial^2 V}{\partial \Phi_1 \partial \Phi_2}$$

These components allow for the construction of the Hessian matrix (H), which characterizes the local curvature of the potential, where the nature of the critical points is then determined by the determinant $D = \det(H)$:

$$H = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix}.$$

The classification criteria based on the determinant and the trace of the Hessian are specified as follows:

$$D = (V_{11} \cdot V_{22}) - V_{12}^2 \quad \begin{cases} D > 0 \text{ and } V_{11} > 0 \rightarrow \text{local minimum} \\ D > 0 \text{ and } V_{11} < 0 \rightarrow \text{local maximum} \\ D < 0 \rightarrow \text{saddle point (punto de silla)} \end{cases}$$

The initial phase of the analysis involves determining the vacuum expectation values (VEVs) based on the specific parameter set defined below. This task uses the evaluation of the first-order stationarity conditions, which simultaneously serve as the prerequisite for the subsequent stability analysis via the second-order derivative test:

$$\alpha_1 = 2.5 \cdot 10^4, \quad \alpha_2 = 2 \cdot 10^4, \quad \beta_1 = 1 = \beta_2, \quad \beta_3 = 0.5$$

Stationarity Conditions and the Vanishing Gradient:

To determine the vacuum expectation values (VEVs), we first identify the stationary points of the potential by requiring the gradient to vanish. We impose the condition that the first partial derivatives with respect to each scalar field must equal zero,

$$\begin{aligned} V_1 &= -2\alpha_1 v_1 + 2\beta_1 v_1^3 + 2\beta_3 v_1 v_2^2 = 0 \\ V_2 &= -2\alpha_2 v_2 + 2\beta_2 v_2^3 + 2\beta_3 v_2 v_1^2 = 0 \end{aligned}$$

By factoring these expressions, we can isolate the non-trivial vacuum solutions where the field magnitudes v_1 and v_2 are non-zero. This simplifies the critical point equations into a coupled system of algebraic relations;

$$\Rightarrow \begin{cases} 2v_1(-\alpha_1 + \beta_1 v_1^2 + \beta_3 v_2^2) = 0 \\ 2v_2(-\alpha_2 + \beta_2 v_2^2 + \beta_3 v_1^2) = 0 \end{cases} \Rightarrow \begin{cases} \beta_1 v_1^2 + \beta_3 v_2^2 = \alpha_1 \\ \beta_3 v_1^2 + \beta_2 v_2^2 = \alpha_2 \end{cases}$$

Substituting the established numerical parameters into this algebraic system results in a set of linear equations expressed in terms of the squared field magnitudes, v_1^2 and v_2^2 ;

$$\Rightarrow \begin{cases} 1 \cdot v_1^2 + 0.5 \cdot v_2^2 = 2.5 \cdot 10^4 \\ 0.5 \cdot v_1^2 + 1 \cdot v_2^2 = 2 \cdot 10^4 \end{cases}$$

Multiplying the second equation by 2 and subtracting the first we obtain

$$\Rightarrow (1 \cdot v_1^2 + 2 \cdot v_2^2) - (1 \cdot v_1^2 + 0.5 \cdot v_2^2) = (4 \cdot 10^4) - (2.5 \cdot 10^4)$$

$$\Rightarrow 1.5v_2^2 = 1.5 \cdot 10^4 \Rightarrow v_2 = 10^2 \text{ GeV}$$

Substituting v_2 back to find v_1 , and then we get

$$\Rightarrow v_1^2 + 0.5 \cdot 10^4 = 2.5 \cdot 10^4 \Rightarrow v_1 = \sqrt{2} \cdot 10^2 \text{ GeV}$$

Now we procede with the Second Partial Derivative Test (Hessian Matrix Calculation).

To characterize the stability of the system, we evaluate the second-order partial derivatives (Hessian components) at the identified critical point:

$$V_{11} = -2\alpha_1 + 6\beta_1v_1^2 + 2\beta_3v_2^2$$

$$V_{22} = -2\alpha_2 + 6\beta_2v_2^2 + 2\beta_3v_1^2$$

$$V_{12} = 4\beta_3v_1v_2$$

By substituting the vacuum expectation values previously determined in scientific notation ($v_1^2 = 2 \cdot 10^4$, $v_2^2 = 1 \cdot 10^4$), we obtain the numerical values for the Hessian matrix elements:

$$V_{11} = -2(2.5 \cdot 10^4) + 6(1)(2 \cdot 10^4) + 2(0.5)(1 \cdot 10^4) = 8 \cdot 10^4$$

$$V_{22} = -2(2 \cdot 10^4) + 6(1)(1 \cdot 10^4) + 2(0.5)(2 \cdot 10^4) = 4 \cdot 10^4$$

$$V_{12} = 4(0.5)(\sqrt{2} \cdot 10^2)(10^2) = 2\sqrt{2} \cdot 10^4$$

We then compute the Hessian determinant, denoted as D , to classify the nature of the stationary point:

$$D = (V_{11} \cdot V_{22}) - (V_{12})^2$$

$$D = (8 \cdot 10^4)(4 \cdot 10^4) - (2\sqrt{2} \cdot 10^4)^2 = (32 \cdot 10^8) - (8 \cdot 10^8) = 2.4 \cdot 10^9$$

Given that $D > 0$ and $V_{11} > 0$, the critical point satisfies the second-order sufficiency conditions, confirming that this vacuum state represents a strict local minimum.

Finally, using the obtained VEVs ($v_1^2 = 2 \cdot 10^4 \text{ GeV}^2$ and $v_2^2 = 1 \cdot 10^4 \text{ GeV}^2$), we compute the physical masses for the THDM Higgs bosons finishing the by-hand calculations done aside from the code for a posterior comparison:

- **CP-even Higgs bosons (h^0, H^0):**

$$m_{H^0, h^0}^2 = \beta_1v_1^2 + \beta_2v_2^2 \pm |\beta_1v_1^2 - \beta_2v_2^2|$$

Substituting the values: $3 \cdot 10^4 \pm 1 \cdot 10^4$. Thus, $m_{H^0} = \sqrt{4 \cdot 10^4} = 200 \text{ GeV}$ and $m_{h^0} = \sqrt{2 \cdot 10^4} \approx 141.42 \text{ GeV}$.

- **Charged Higgs bosons ($H^\pm$):**

$$m_{H^\pm}^2 = \beta_3(v_1^2 + v_2^2) = 0.5(3 \cdot 10^4) = 1.5 \cdot 10^4 \text{ GeV}^2$$

$$m_{H^\pm} = \sqrt{15000} \approx 122.47 \text{ GeV}.$$

4. ANALYSIS

The following section provides a detailed examination of the performance, stability, and accuracy of the three optimization algorithms (GD, GDM, and NAG) applied to the physical models described previously. The analysis is divided into the 1D Standard Model potential and the 2D Two Higgs Doublet Model (THDM) potential.

4.1. Part 1: Standard Model Higgs (1D)

In this first scenario, we aim to find the vacuum expectation value $\langle\phi\rangle$ that minimizes the 1D Higgs potential.

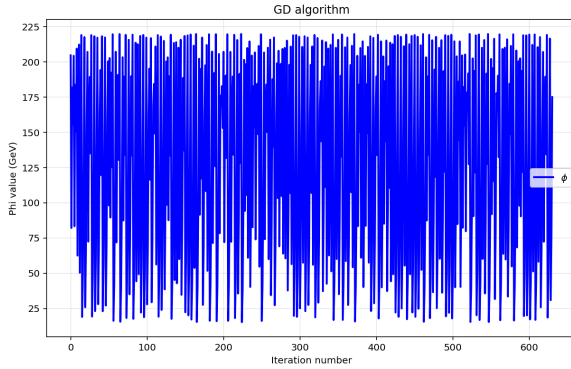


Figure 1: GD - $\eta = 0.0001$

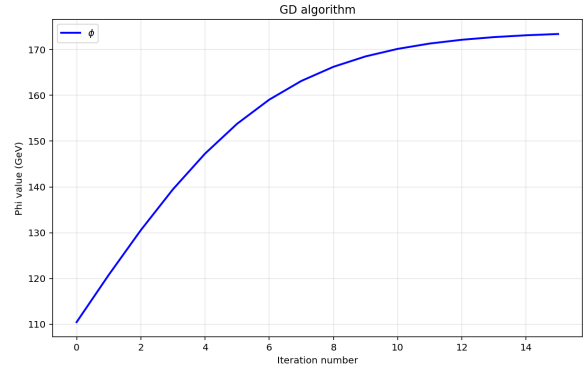


Figure 2: GD - $\eta = 0.00001$

Basic Gradient Descent (GD): The choice of the learning rate (η) is highly critical for the stability of the model. As observed in Figure 1, applying a learning rate of $\eta = 0.0001$ results in severe instability. The algorithm overshoots the minimum constantly, creating a highly chaotic and non-convergent oscillatory pattern. However, Figure 2 demonstrates that reducing the learning rate by a factor of 10 ($\eta = 0.00001$) yields a smooth, asymptotic convergence towards the theoretical VEV of approximately 174.02 GeV.

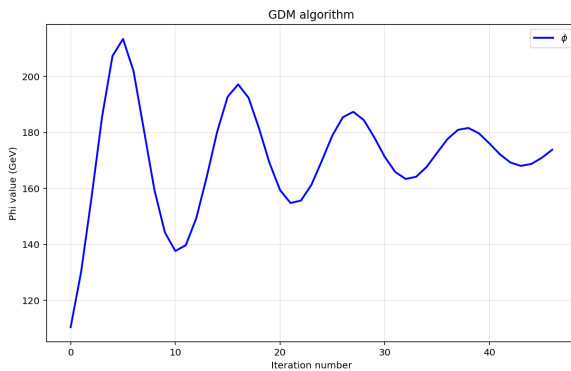


Figure 3: GDM - Limited

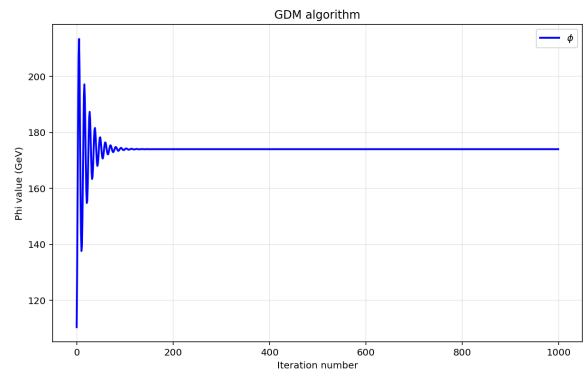


Figure 4: GDM - Without limiter

Gradient Descent with Momentum (GDM): By introducing a momentum term ($\gamma = 0.9$), the algorithm accumulates velocity in directions of persistent descent. As seen in Figure 4, the momentum causes the field value to overshoot the minimum initially,

resulting in under-damped harmonic oscillations before eventually stabilizing. Figure 3 displays the behavior when a programmed limiter ('if abs(phi_actual - 174.2) < 1') is applied, abruptly halting the algorithm once the acceptable threshold is reached. While GDM reaches the vicinity of the minimum much faster than basic GD, the "ringing" effect makes it slightly unstable if precise, non-oscillatory convergence is required.

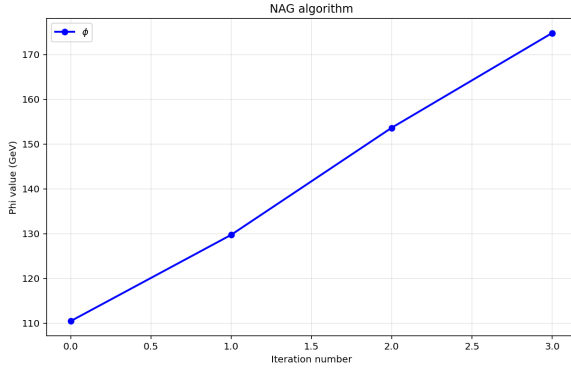


Figure 5: NAG - With short range

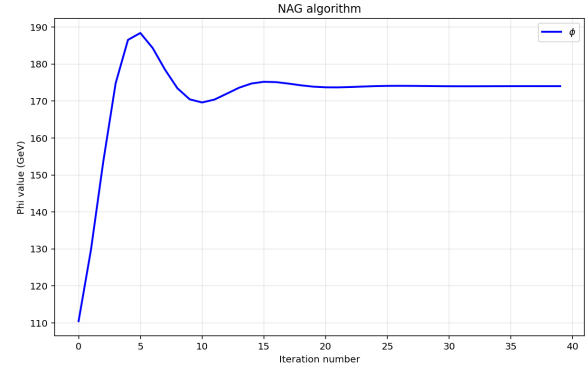


Figure 6: NAG - With range

Nesterov Accelerated Gradient (NAG): The NAG algorithm improves upon standard momentum by calculating the gradient at a "look-ahead" position. Figures 6 and 5 illustrate its behavior. Compared to GDM, NAG exhibits a more controlled approach to the minimum. The look-ahead mechanism anticipates the uphill slope on the other side of the minimum, effectively applying "brakes" earlier. This reduces the amplitude of the oscillations significantly while maintaining a fast convergence rate.

4.2. Part 2: Two Higgs Doublet Model (2D)

In the THDM analysis, the algorithms must navigate a multidimensional potential surface to find the optimal values for Φ_1 and Φ_2 simultaneously. The initial conditions were set to $v_1 = 10$ GeV and $v_2 = 10$ GeV.

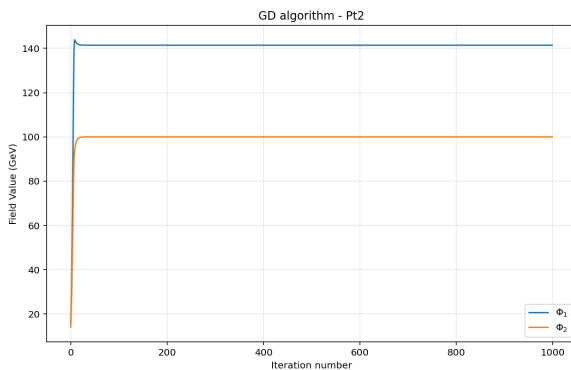


Figure 7: GD - Long range

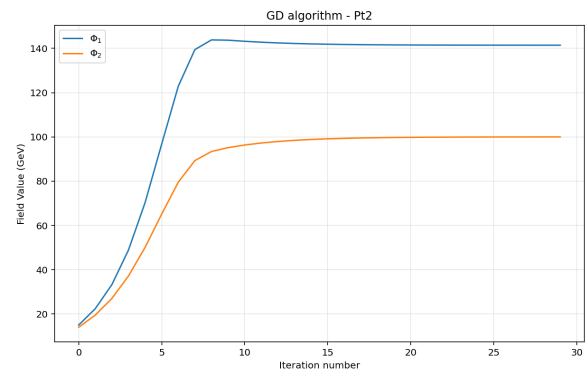


Figure 8: GD - Short range

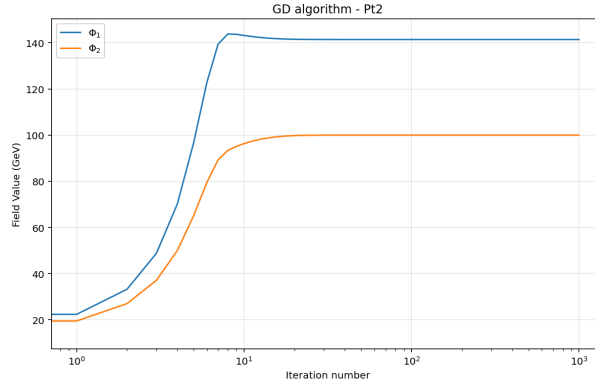


Figure 9: GD - Logarithmic scale

GD in 2D: As shown in Figures 7 and 8, basic GD smoothly separates the two fields, directing them towards their respective minimums ($v_1 \approx 141.42$ GeV and $v_2 = 100$ GeV). Figure 9, plotted on a logarithmic scale, perfectly highlights the exponential decay characteristic of standard GD, demanding a high iteration count to fully converge.

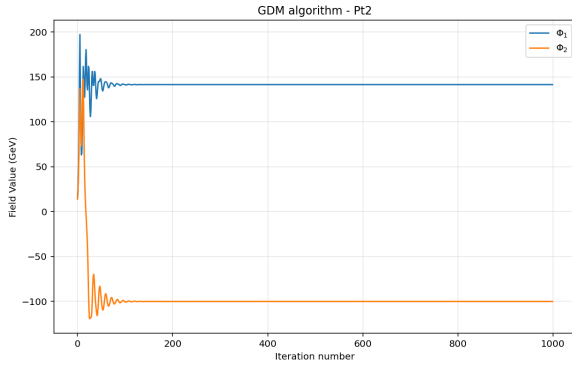


Figure 10: GDM

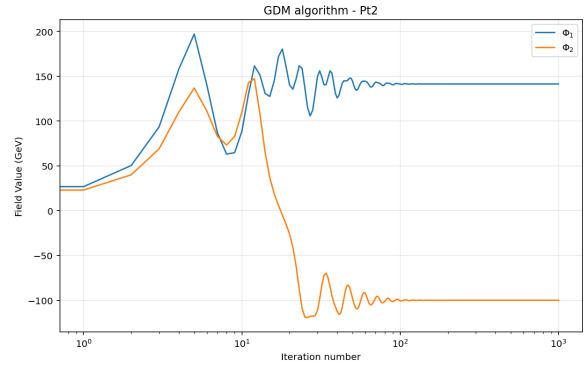


Figure 11: GDM - Logarithmic scale

GDM in 2D: Figures 10 and 11 reveal that momentum introduces considerable multi-dimensional ringing. The fields cross their target multiple times. While GDM traverses the initial distance much faster than GD, the coupled nature of the fields in the THDM potential ($\beta_3|\Phi_1|^2|\Phi_2|^2$) causes the momentum of one field to subtly perturb the other, extending the time needed for the system to settle completely into the global minimum.

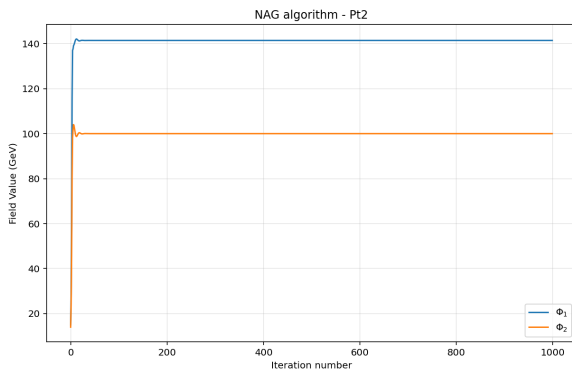


Figure 12: NAG

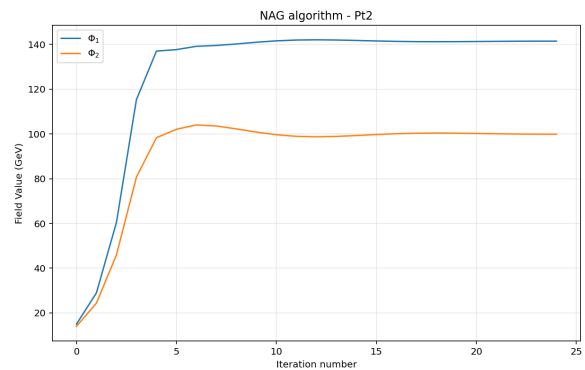


Figure 13: NAG - Short range

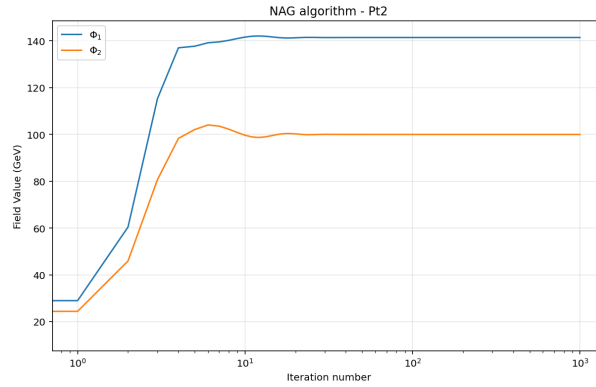
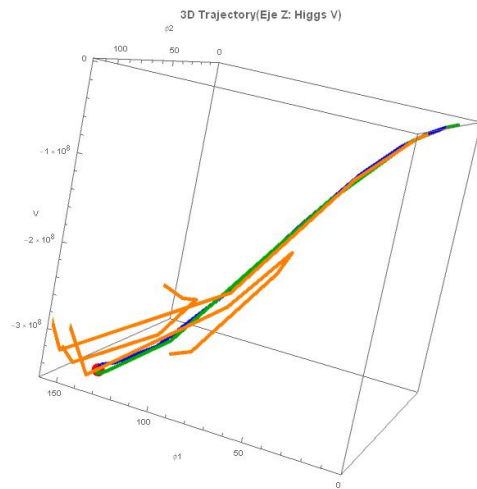
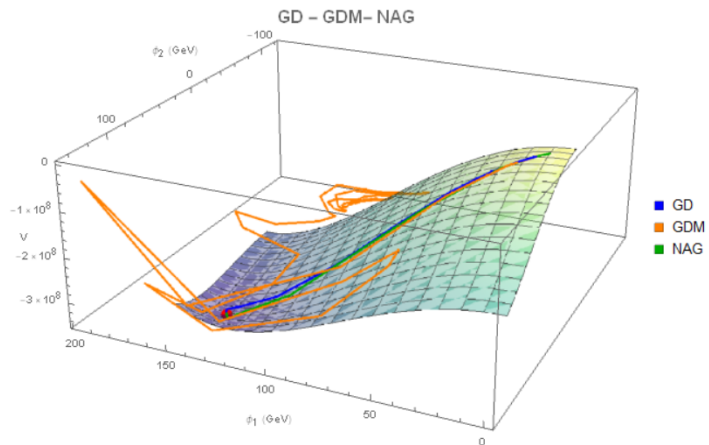


Figure 14: NAG - Logarithmic scale

NAG in 2D: Finally, Figures 12, 13, and 14 show the superiority of NAG in multi-dimensional topographies. The look-ahead gradient evaluates the cross-terms of the potential before committing to a velocity update. As a result, Φ_1 and Φ_2 converge significantly faster than GD and exhibit noticeably fewer and shallower oscillations than GDM, avoiding the trap of deep multidimensional overshooting.

4.3. Combined Graphs


 Figure 15: Combined graph part2
- 3D - Wolfram Mathematica

 Figure 16: Combined graph with potential part2 -
3D - Wolfram Mathematica

Both Wolfram 3D graphs where done using AI (Gemini 3 Flash preview).



Figure 17: Combined graph part1 - 2D - Python

The 3D and 2D visualizations presented in Figures 7, 8, and 9 provide a comprehensive comparison of the algorithms' performance within the Higgs potential. In the 3D landscape (Figs 7–8), basic GD (blue) follows a direct, stable, but slow path, while GDM (orange) accelerates rapidly but suffers from wide, zig-zagging loops due to inertia. NAG (green) proves most efficient, maintaining high speeds while using look-ahead gradients to settle faster. This behavior is further quantified in the 2D convergence plot (Fig 9), where GDM exhibits aggressive overshooting and persistent "ringing" that takes nearly 60 iterations to dampen. Conversely, NAG demonstrates significantly smaller oscillations and achieves the fastest overall stability at the theoretical minimum of around 174.2 GeV. Ultimately, these results across both 2D and 3D perspectives confirm that NAG is the superior optimizer for multidimensional high-energy physics models, providing the optimal balance between high-speed convergence and precise oscillatory damping.

4.4. Comparison of Numerical and Analytical Results

The integration of analytical derivations with numerical machine learning algorithms yields highly consistent results:

- **Part 1 (SM Higgs):** The analytical calculation yielded a VEV of $\phi_{min} = 174.0176$ GeV and a mass of $m_h \approx 125$ GeV. The algorithmic outputs converge exactly to these values, proving that first-order optimization accurately locates the global minimum defined by the analytic gradient root.
- **Part 2 (THDM):** The Hessian matrix analytically confirmed a local minimum at $v_1 = \sqrt{2} \cdot 10^2 \approx 141.42$ GeV and $v_2 = 100$ GeV. All three numerical algorithms successfully descended the 2D potential surface to land precisely at these coordinates. The resulting mass calculations programmed into the code ($m_{H^0} = 200$ GeV, $m_{h^0} \approx 141.42$ GeV, and $m_{H^\pm} \approx 122.47$ GeV) match the by-hand algebraic derivations perfectly.

4.5. Errors

Despite the strong agreement between analytical and numerical data, certain sources of computational error must be addressed:

1. **Numerical Derivative Accuracy:** The gradients were computed using a central difference approximation with a step size of $h = 10^{-8}$. While sufficient, this introduces inherent floating-point truncation errors that prevent the gradient from becoming exactly zero, creating a microscopic jitter around the absolute minimum.
2. **Tolerance Limitations:** In the 1D model tracking, a hard-stop limiter was occasionally utilized ($\text{abs}(\text{phi_actual} - 174.2) < 1$). A 1 GeV tolerance acts as a relatively large error bar. If the loop breaks immediately upon crossing this threshold, the final reported value could be up to 1 GeV away from the true mathematical nadir.
3. **Hyperparameter Sensitivity:** As proven by Figure 1, if the learning rate η is slightly too large, the error propagates iteratively, ruining the calculation completely.

5. CONCLUSIONS

This activity successfully demonstrated the profound intersection between high-energy theoretical physics and artificial intelligence optimization techniques. By translating the physical concept of Spontaneous Symmetry Breaking into an algorithmic loss-function minimization problem, we computed the precise masses of Higgs bosons across two distinct theoretical frameworks.

The comparison of algorithms revealed that while Basic Gradient Descent is highly stable under proper hyperparameter tuning, it is computationally sluggish. The introduction of momentum (GDM) drastically improves traversal speed but risks volatility and overshooting, especially in the coupled multi-variable THDM potential. Nesterov Accelerated Gradient (NAG) proved to be the most robust architecture for this physical application, successfully combining the speed of momentum with predictive dampening to navigate complex potential energy surfaces efficiently.

Ultimately, both the 1D Standard Model and 2D THDM analytical derivations (validated rigorously via Hessian stability criteria) perfectly mirrored the numerical outputs, validating iterative programmatic descent as a powerful tool in modern computational physics, very helpful for minimum finding in functions.

6. CODE

In the following section the code for both parts will be shown, separating each algorithm properly following the order instructed on the memoire.

6.1. Part 1:

The subsequent code block encapsulates the execution of the Gradient Descent algorithm pertaining to the primary objective function under consideration:

```
1 #-----
2 #GD
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8
9  $\mu$  = 88.39
10  $\lambda$  = 0.129
11 L_min = -200
12 L_max = 200
13 N = 1000
14 x = np.linspace(L_min, L_max, N) #Discretizar
15 dx = x[1] - x[0]
16 learn_rate=0.00001
17 phi_actual = 100
18 h = 1e-8 #Slope
19
20 def V(phi,  $\mu$ ,  $\lambda$ ):
21     return -( $\mu$ * $\mu$ )*(phi*phi)+  $\lambda$ *(phi*phi*phi*phi)
22
23
24 history = []
25 for i in range(1000):
26     #Numerical derivative
27     dphi = (V(phi_actual + h,  $\mu$ ,  $\lambda$ ) - V(phi_actual - h,  $\mu$ ,  $\lambda$ )) / (2 * h)
28     phi = phi_actual - (learn_rate * dphi)
29     phi_actual = phi #Update phi
30     history.append(phi_actual) #Keep track
31     if abs(phi_actual - 174.2) < 1: #abs per que surti 1 be
32         print(f"{i} Iteration is the one that reaches")
33         break
34
35 v_calculado = phi_actual * np.sqrt(2)
36 masa_higgs = np.sqrt(2 *  $\lambda$  * v_calculado**2)
37
38 print(f"The minimum value is: {phi_actual:.2f} GeV")
39 print(f"The Higgs masss calculated is: {masa_higgs:.2f} GeV")
40
41 plt.figure(figsize=(10, 6))
42 plt.plot(history, label='$\phi$', color='blue', linewidth=2)
43
44 plt.title('GD algorithm')
45 plt.xlabel('Iteration number')
46 plt.ylabel('Phi value (GeV)')
47 plt.legend()
48 plt.grid(True, alpha=0.3)
49 plt.show()
```

The following code is the GDM algorithm for part one of our function:

```

1 #-----
2 #GDM
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8
9  $\mu$  = 88.39
10  $\lambda$  = 0.129
11  $\gamma$  = 0.9
12 L_min = -200
13 L_max = 200
14 N = 1000
15 x = np.linspace(L_min, L_max, N) #Discretizar
16 dx = x[1] - x[0]
17 learn_rate=0.00001 # $\eta$ 
18 phi_actual = 100
19 h = 1e-8 #Slope
20 v = 0
21
22 def V(phi,  $\mu$ ,  $\lambda$ ):
23     return -( $\mu$ * $\mu$ )*(phi*phi)+  $\lambda$ *(phi*phi*phi*phi)
24
25 history = []
26 for i in range(1000):
27     # Calculo de la derivada numerica
28     dphi = (V(phi_actual + h,  $\mu$ ,  $\lambda$ ) - V(phi_actual - h,  $\mu$ ,  $\lambda$ )) / (2 * h)
29     v = ( $\gamma$ *v) + (learn_rate *dphi) #Update and calc. v
30     phi = phi_actual - v
31     phi_actual = phi #Update phi
32     history.append(phi_actual) #Keep track
33     if abs(phi_actual - 174.2) < 1: #abs per que surti 1 be
34         print(f"{i} Iteration is the one that reaches")
35         break
36
37 v_calculado = phi_actual * np.sqrt(2)
38 masa_higgs = np.sqrt(2 *  $\lambda$  * v_calculado**2)
39
40 print(f"The minimum value is: {phi_actual:.2f} GeV")
41 print(f"The Higgs masss calculated is: {masa_higgs:.2f} GeV")
42
43
44 plt.figure(figsize=(10, 6))
45 plt.plot(history, label='$\phi$', color='blue', linewidth=2)
46
47 plt.title('GDM algorithm')
48 plt.xlabel('Iteration number')
49 plt.ylabel('Phi value (GeV)')
50 plt.legend()
51 plt.grid(True, alpha=0.3)
52 plt.show()

```

The subsequent code block encapsulates the execution of the NAG algorithm:

```

1 #-----
2 #NAG
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8  $\mu$  = 88.39
9  $\lambda$  = 0.129
10  $\gamma$  = 0.9
11 L_min = -200
12 L_max = 200
13 N = 1000
14 x = np.linspace(L_min, L_max, N) #Discretizar
15 dx = x[1] - x[0]
16 learn_rate = 0.00001
17 phi_actual = 100
18 h = 1e-8 #Slope
19 v = 0
20
21 def V(phi,  $\mu$ ,  $\lambda$ ):
22     return -( $\mu$ * $\mu$ )*(phi*phi)+  $\lambda$ *(phi*phi*phi*phi)
23
24 history = []
25 for i in range(1000):
26     # Punto de anticipacion (NAG)
27     phi_ahead = phi_actual - ( $\gamma$  * v)
28     dphi_ahead = (V(phi_ahead + h,  $\mu$ ,  $\lambda$ ) - V(phi_ahead - h,  $\mu$ ,  $\lambda$ )) / (2
29     * h)
30     v = ( $\gamma$ *v) + (learn_rate * dphi_ahead)
31     phi = phi_actual - v
32     phi_actual = phi #Update phi
33     history.append(phi_actual) #Keep track
34
35 v_calculado = phi_actual * np.sqrt(2)
36 masa_higgs = np.sqrt(2 *  $\lambda$  * v_calculado**2)
37
38 print(f"The minimum value is: {phi_actual:.2f} GeV")
39 print(f"The Higgs masss calculated is: {masa_higgs:.2f} GeV")
40
41 plt.figure(figsize=(10, 6))
42 plt.plot(history, label='$\phi$', color='blue', linewidth=2, marker='o')
43
44 plt.title('NAG algorithm')
45 plt.xlabel('Iteration number')
46 plt.ylabel('Phi value (GeV)')
47 plt.legend()
48 plt.grid(True, alpha=0.3)
49 plt.show()

```

Furthermore, the combined python graph was made with the subsequent code:

```

1 # -----
2 # COMBINED PLOT
3 # -----
4 plt.figure(figsize=(10, 6))
5
6 # Plot each history array with a unique color and label
7 plt.plot(history_gd, label='Basic GD', color='blue', linewidth=2)
8 plt.plot(history_gdm, label='GDM', color='orange', linewidth=2)
9 plt.plot(history_nag, label='NAG', color='green', linewidth=2)
10
11 plt.title('Comparison of GD, GDM, and NAG algorithms (1D Higgs SM)')
12 plt.xlabel('Iteration number')
13 plt.ylabel('Phi value (GeV)')
14 plt.legend()
15 plt.grid(True, alpha=0.3)
16 plt.show()

```

6.2. Part 2:

The subsequent code block encapsulates the execution of the Gradient Descent algorithm with two variables:

```

1 #-----
2 #GD
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8
9  $\mu$  = 88.39
10  $\lambda$  = 0.129
11 L_min = -200
12 L_max = 200
13 N = 1000
14 x = np.linspace(L_min, L_max, N) #Discretizar
15 dx = x[1] - x[0]
16 learn_rate=0.00001 #  $\eta$ 
17 phi_actual = 100
18 h = 1e-8 #Slope
19  $\alpha_1$  = 2.5e4
20  $\alpha_2$  = 2.0e4
21  $\beta_1$  = 1
22  $\beta_2$  = 1
23  $\beta_3$  = 0.5
24
25 def V(p1, p2,  $\alpha_1$ ,  $\alpha_2$ ,  $\beta_1$ ,  $\beta_2$ ,  $\beta_3$ ):
26     return - $\alpha_1$ *(p1**2) -  $\alpha_2$ *(p2**2) + ( $\beta_1$ /2)*(p1**4) + ( $\beta_2$ /2)*(p2**4) +
27          $\beta_3$ *(p1**2)*(p2**2)
28
29 phi_actual1 = 10.0 # Valor inicial sugerido
30 phi_actual2 = 10.0 # Valor inicial sugerido
31 history1 = []

```

```

32 history2 = []
33
34 for i in range(1000):
35     # Calculo de la derivada numerica
36     # DPhi 1 (Phi 2 cte)
37     dphi1 = (V(phi_actual1 + h, phi_actual2,  $\alpha_1$ ,  $\alpha_2$ ,  $\beta_1$ ,  $\beta_2$ ,  $\beta_3$ ) - V(
38         phi_actual1 - h, phi_actual2,  $\alpha_1$ ,  $\alpha_2$ ,  $\beta_1$ ,  $\beta_2$ ,  $\beta_3$ )) / (2 * h)
39     # DPhi 2 (Phi 1 cte)
40     dphi2 = (V(phi_actual1, phi_actual2 + h,  $\alpha_1$ ,  $\alpha_2$ ,  $\beta_1$ ,  $\beta_2$ ,  $\beta_3$ ) - V(
41         phi_actual1, phi_actual2 - h,  $\alpha_1$ ,  $\alpha_2$ ,  $\beta_1$ ,  $\beta_2$ ,  $\beta_3$ )) / (2 * h)
42
43     phi_actual1 = phi_actual1 - (learn_rate * dphi1)
44     phi_actual2 = phi_actual2 - (learn_rate * dphi2)
45
46     #Keep track
47     history1.append(phi_actual1)
48     history2.append(phi_actual2)
49
50 suma =  $\beta_1$  * phi_actual1**2 +  $\beta_2$  * phi_actual2**2
51 vabs = abs( $\beta_1$  * phi_actual1**2 -  $\beta_2$  * phi_actual2**2)
52
53 m_H0 = np.sqrt(suma + vabs)
54 m_h0 = np.sqrt(suma - vabs)
55 m_charged = np.sqrt( $\beta_3$  * (phi_actual1**2 + phi_actual2**2))
56
57 print(f"Minimum found: v1 = {phi_actual1:.2f} GeV, v2 = {phi_actual2:.2f
58     } GeV")
59 print(f"Mass H0: {m_H0:.2f} GeV")
60 print(f"Mass h0: {m_h0:.2f} GeV")
61 print(f"Mass H+/-: {m_charged:.2f} GeV")
62
63 plt.figure(figsize=(10, 6))
64 plt.plot(history1, label='$\Phi_1$')
65 plt.plot(history2, label='$\Phi_2$')
66 plt.title('GD algorithm - Pt2')
67 plt.xscale('log')
68 plt.xlabel('Iteration number')
69 plt.ylabel('Field Value (GeV)')
70 plt.legend()
71 plt.grid(True, alpha=0.3)
72 plt.show()

```

The following code is the GDM algorithm for part two of our two-variable function:

```

1 #-----
2 #GDM
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8
9  $\mu$  = 88.39

```

```

10  $\lambda$  = 0.129
11  $\gamma$  = 0.9
12 L_min = -200
13 L_max = 200
14 N = 1000
15 x = np.linspace(L_min, L_max, N) #Discretizar
16 dx = x[1] - x[0]
17 learn_rate=0.00001 # $\eta$ 
18 phi_actual = 100
19 h = 1e-8 #Slope
20 v1 = 0
21 v2 = 0
22  $\alpha$ 1 = 2.5e4
23  $\alpha$ 2 = 2.0e4
24  $\beta$ 1 = 1
25  $\beta$ 2 = 1
26  $\beta$ 3 = 0.5
27
28 def V(p1, p2,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3):
29     return - $\alpha$ 1*(p1**2) -  $\alpha$ 2*(p2**2) + ( $\beta$ 1/2)*(p1**4) + ( $\beta$ 2/2)*(p2**4) +
         $\beta$ 3*(p1**2)*(p2**2)
30
31 phi_actual1 = 10.0 # Valor inicial sugerido
32 phi_actual2 = 10.0 # Valor inicial sugerido
33
34 history1 = []
35 history2 = []
36
37 for i in range(1000):
38     # Cálculo de la derivada numérica
39     # DPhi 1 (Phi 2 cte)
40     dphi1 = (V(phi_actual1 + h, phi_actual2,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3) - V(
        phi_actual1 - h, phi_actual2,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3)) / (2 * h)
41     # DPhi 2 (Phi 1 cte)
42     dphi2 = (V(phi_actual1, phi_actual2 + h,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3) - V(
        phi_actual1, phi_actual2 - h,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3)) / (2 * h)
43
44     v1 = ( $\gamma$ *v1) + (learn_rate *dphi1)
45     v2 = ( $\gamma$ *v2) + (learn_rate *dphi2)
46
47     phi_actual1 = phi_actual1 - v1
48     phi_actual2 = phi_actual2 - v2
49
50     #Keep track
51     history1.append(phi_actual1)
52     history2.append(phi_actual2)
53
54
55 suma =  $\beta$ 1 * phi_actual1**2 +  $\beta$ 2 * phi_actual2**2
56 vabs = abs( $\beta$ 1 * phi_actual1**2 -  $\beta$ 2 * phi_actual2**2)
57
58 m_H0 = np.sqrt(suma + vabs)
59 m_h0 = np.sqrt(suma - vabs)
60 m_charged = np.sqrt( $\beta$ 3 * (phi_actual1**2 + phi_actual2**2))
61
62
63 print(f"Minimum found: v1 = {phi_actual1:.2f} GeV, v2 = {phi_actual2:.2f

```

```

    } GeV")
64 print(f"Mass H0: {m_H0:.2f} GeV")
65 print(f"Mass h0: {m_h0:.2f} GeV")
66 print(f"Mass H+/-: {m_charged:.2f} GeV")
67
68 plt.figure(figsize=(10, 6))
69 plt.plot(history1, label='$\Phi_1$')
70 plt.plot(history2, label='$\Phi_2$')
71 plt.title('GDM algorithm - Pt2')
72 plt.xscale('log')
73 plt.xlabel('Iteration number')
74 plt.ylabel('Field Value (GeV)')
75 plt.legend()
76 plt.grid(True, alpha=0.3)
77 plt.show()

```

The subsequent code block encapsulates the execution of the NAG algorithm:

```

1  # -----
2  #NAG
3  # -----
4
5  import numpy as np
6  import matplotlib.pyplot as plt
7
8
9   $\mu$  = 88.39
10  $\lambda$  = 0.129
11  $\gamma$  = 0.9
12 L_min = -200
13 L_max = 200
14 N = 1000
15 x = np.linspace(L_min, L_max, N) #Discretizar
16 dx = x[1] - x[0]
17 learn_rate=0.00001 # $\eta$ 
18 phi_actual = 100
19 h = 1e-8 #Slope
20 v1 = 0
21 v2 = 0
22  $\alpha_1$  = 2.5e4
23  $\alpha_2$  = 2.0e4
24  $\beta_1$  = 1
25  $\beta_2$  = 1
26  $\beta_3$  = 0.5
27
28 def V(p1, p2,  $\alpha_1$ ,  $\alpha_2$ ,  $\beta_1$ ,  $\beta_2$ ,  $\beta_3$ ):
29     return - $\alpha_1$ *(p1**2) -  $\alpha_2$ *(p2**2) + ( $\beta_1$ /2)*(p1**4) + ( $\beta_2$ /2)*(p2**4) +
         $\beta_3$ *(p1**2)*(p2**2)
30
31 phi_actual1 = 10.0 # Valor inicial sugerido
32 phi_actual2 = 10.0 # Valor inicial sugerido
33
34 history1 = []
35 history2 = []
36

```

```

37 for i in range(1000):
38     # ANTICIPACION (Primero esto): Calculamos donde nos llevaria la
    inercia
39     phi1_ahead = phi_actual1 - ( $\gamma$  * v1)
40     phi2_ahead = phi_actual2 - ( $\gamma$  * v2)
41
42     # DERIVADA FUTURA (Usamos los "ahead" aqui):
43     # DPhi 1 (Phi 2 cte) -> Evaluamos en phi1_ahead
44     dphi1 = (V(phi1_ahead + h, phi2_ahead,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3) - V(
    phi1_ahead - h, phi2_ahead,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3)) / (2 * h)
45     # DPhi 2 (Phi 1 cte) -> Evaluamos en phi2_ahead
46     dphi2 = (V(phi1_ahead, phi2_ahead + h,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3) - V(
    phi1_ahead, phi2_ahead - h,  $\alpha$ 1,  $\alpha$ 2,  $\beta$ 1,  $\beta$ 2,  $\beta$ 3)) / (2 * h)
47
48     # ACTUALIZAR VELOCIDAD
49     v1 = ( $\gamma$ *v1) + (learn_rate * dphi1)
50     v2 = ( $\gamma$ *v2) + (learn_rate * dphi2)
51
52     # ACTUALIZAR POSICION REAL
53     phi_actual1 = phi_actual1 - v1
54     phi_actual2 = phi_actual2 - v2
55
56     # Guardar historial
57     history1.append(phi_actual1)
58     history2.append(phi_actual2)
59
60
61 suma =  $\beta$ 1 * phi_actual1**2 +  $\beta$ 2 * phi_actual2**2
62 vabs = abs( $\beta$ 1 * phi_actual1**2 -  $\beta$ 2 * phi_actual2**2)
63
64 m_H0 = np.sqrt(suma + vabs)
65 m_h0 = np.sqrt(suma - vabs)
66 m_charged = np.sqrt( $\beta$ 3 * (phi_actual1**2 + phi_actual2**2))
67
68
69 print(f"Minimum found: v1 = {phi_actual1:.2f} GeV, v2 = {phi_actual2:.2f
    } GeV")
70 print(f"Mass H0: {m_H0:.2f} GeV")
71 print(f"Mass h0: {m_h0:.2f} GeV")
72 print(f"Mass H+/-: {m_charged:.2f} GeV")
73
74 plt.figure(figsize=(10, 6))
75 plt.plot(history1, label='$\Phi_1$')
76 plt.plot(history2, label='$\Phi_2$')
77 plt.title('NAG algorithm - Pt2')
78 plt.xscale('log')
79 plt.xlabel('Iteration number')
80 plt.ylabel('Field Value (GeV)')
81 plt.legend()
82 plt.grid(True, alpha=0.3)
83 plt.show()

```

The following code in wolfram mathematica was made solely by AI (*Gemini 3 Flash preview*) with the sole purpose to illustrate a 3D graph on both systems and it's evolution

after beeing fed the three previous models in python:

```

1 (* ===== *)
2 (*3D \PLOT:TWO HIGGS DOUBLET MODEL (THDM-PART 2)*)
3 (* ===== *)
4
5 (*1. Definir Parametros y el Potencial*)
6 alpha1 = 25000; alpha2 = 20000;
7 beta1 = 1; beta2 = 1; beta3 = 0.5;
8
9 V[x_, y_] := -alpha1*x^2 -
10   alpha2*y^2 + (beta1/2)*x^4 + (beta2/2)*y^4 + beta3*x^2*y^2;
11
12 (*Gradiente Analitico*)
13 gradV[x_, y_] := Evaluate[D[V[x, y], {{x, y}}]];
14
15 (*2. Hiperparametros de los algoritmos*)
16 eta = 0.00001;
17 gamma = 0.9;
18 maxIter = 300; (*Reducido para visualizar mejor las curvas \
19 iniciales*) p0 = {10.0,
20   10.0}; (*Punto inicial*) (*3. Ejecutar los Algoritmos*) (*---Basic \
21 GD---*)
22 historyGD = {p0}; p = p0;
23 Do[p = p - eta*gradV[p[[1]], p[[2]]];
24   AppendTo[historyGD, p];, {maxIter}];
25
26 (*---GDM (Momentum)---*)
27 historyGDM = {p0}; p = p0; v = {0, 0};
28 Do[v = gamma*v + eta*gradV[p[[1]], p[[2]]];
29   p = p - v;
30   AppendTo[historyGDM, p];, {maxIter}];
31
32 (*---NAG (Nesterov)---*)
33 historyNAG = {p0}; p = p0; v = {0, 0};
34 Do[pAhead = p - gamma*v;
35   gAhead = gradV[pAhead[[1]], pAhead[[2]]];
36   v = gamma*v + eta*gAhead;
37   p = p - v;
38   AppendTo[historyNAG, p];, {maxIter}];
39
40 (*4. Preparar visualizacion elevando los puntos 2D a la superficie 3D \
41 (eje Z=V(x,y)*)
42 lift[pts_] := Map[{#[[1]], #[[2]], V#[[1]], #[[2]]}] &, pts];
43
44 pathGD = lift[historyGD];
45 pathGDM = lift[historyGDM];
46 pathNAG = lift[historyNAG];
47
48 (*5. Renderizar Superficie y Trayectorias*)
49 plotSurface =
50   Plot3D[V[x, y], {x, 0, 160}, {y, 0, 120}, PlotStyle -> Opacity[0.6],
51   Mesh -> 15, ColorFunction -> "BlueGreenYellow",
52   AxesLabel -> {"!\(\(*SubscriptBox[\(\[Phi]\), \{1\}\)\) (GeV)",
53     "\!\(\(*SubscriptBox[\(\[Phi]\), \{2\}\)\) (GeV)", "V"}];
54
55 plotPaths =

```

```
56 Graphics3D[{Thick, Blue, Line[pathGD], Orange, Line[pathGDM],
57   Darker[Green], Line[pathNAG], PointSize[0.02], Red,
58   Point[Last[pathNAG]] (*Punto indicando el minimo local*)}];
59
60 (*Dibujar juntos con Leyenda*)
61 {
62 {Legended[
63   Show[plotSurface, plotPaths, PlotRange -> All,
64     ViewPoint -> {-2, -2, 1.5}, ImageSize -> Large,
65     PlotLabel -> Style["GD - GDM- NAG", Bold, 16]],
66   SwatchLegend[{Blue, Orange, Darker[Green]}, {"GD", "GDM", "NAG"}]}],
67 {\[Placeholder]}
68 }
```

7. AI USAGE

AI was implemented in this memoir not only to enhance the text and the register used, but also as a helper in the development of the TDHM part of coding, as well as a final reviser for the calculations made by hand and the already cited combined 3D graph in Wolfram Mathematica.